



Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica applicata

Corso di Laurea Triennale in Ingegneria Informatica

Relazione Tecnica d'Esame

FitTrackPro: Local Fitness Tracker

Luglio 2026

Studenti (Gruppo 3):

Marco Giraulo	Mat. 0612708887
Carmine Giudice	Mat. 0612709167
Luca Migliaccio	Mat. 0612709613
Fabio Concilio	Mat. 0612708899
Francesco Mastrogiacono	Mat. 0612708900

Anno Accademico 2025/2026

Indice

1	Descrizione dell'app	2
2	Requisiti	2
2.1	Funzionalità implementate	2
2.2	Ricerca, filtri e organizzazione	3
2.3	Feature avanzate implementate	3
2.4	Funzionalità considerate ma non implementate	4
3	Progettazione dell'app	4
3.1	Struttura generale	4
3.2	Schermate principali e flusso di navigazione	5
3.3	Organizzazione dei dati	6
3.4	Mockup e schermate dell'interfaccia	8
4	Scelte tecnologiche	13
4.1	Framework utilizzato	13
4.2	Librerie principali	13
4.3	Persistenza locale	14
4.4	Motivazione delle scelte effettuate	14
5	Implementazione	14
5.1	Organizzazione del codice e Architettura Software	14
5.2	Gestione dello stato	16
5.3	Gestione della navigazione	16
5.4	Gestione dei dati persistenti	16
5.5	Inserimento, modifica e cancellazione	16
5.6	Validazione e controllo degli input	17
5.7	Gestione delle date	18
5.8	Riepiloghi e statistiche	18
5.9	Aggiornamenti automatici tra entità	19
6	Limitazioni note	19
7	Istruzioni per l'esecuzione	19
8	Verifiche effettuate	20
9	Conclusioni	20

1 Descrizione dell'app

FitTrackPro è un'app mobile sviluppata in React Native con Expo. L'obiettivo dell'app è aiutare l'utente a organizzare la propria attività fisica attraverso la gestione di esercizi, schede di allenamento, sessioni pianificate, allenamenti svolti e obiettivi personali.

L'app è pensata per utenti che si allenano in autonomia e vogliono uno strumento semplice per pianificare gli allenamenti, registrare ciò che è stato svolto e controllare i progressi nel tempo. Non è progettata come piattaforma professionale per personal trainer o palestre, ma come tracker personale locale, utilizzabile anche senza connessione a Internet.

Il problema principale affrontato è la frammentazione delle informazioni relative all'allenamento. Molto spesso esercizi, schede, sessioni e progressi vengono salvati in note separate o in modo poco strutturato. FitTrackPro raccoglie questi dati in sezioni coerenti e permette all'utente di consultarli, modificarli e collegarli tra loro.

Gli scenari d'uso principali sono:

- creare un archivio personale di esercizi;
- costruire schede di allenamento divise per giorni;
- associare a ogni giorno esercizi, serie e ripetizioni;
- pianificare sessioni future partendo da una scheda;
- indicare se una sessione è da svolgere, completata o saltata;
- registrare gli allenamenti effettivamente svolti;
- salvare il carico usato per i singoli esercizi;
- creare obiettivi personali e seguirne l'avanzamento;
- registrare rapidamente l'allenamento previsto per il giorno corrente;
- consultare una dashboard con riepiloghi, costanza settimanale e timer di recupero.

2 Requisiti

2.1 Funzionalità implementate

FitTrackPro implementa le principali funzionalità richieste dalla traccia progettuale.

La sezione **Esercizi** permette di visualizzare l'elenco degli esercizi, consultare il dettaglio, aggiungere nuovi esercizi, modificarli ed eliminarli. Per ogni esercizio vengono gestiti nome, descrizione, gruppo muscolare principale, gruppi secondari, difficoltà, attrezzatura, serie consigliate, ripetizioni consigliate, durata stimata e note. Il gruppo muscolare principale e i gruppi secondari vengono scelti da liste controllate, così da evitare valori incoerenti o scritti in modo diverso.

La sezione **Schede** permette di creare, modificare, eliminare e duplicare schede di allenamento. Ogni scheda contiene nome, descrizione, obiettivo, livello, durata prevista, frequenza settimanale, recupero generale, note e una programmazione divisa per giorni. Ogni giorno contiene esercizi collegati all'archivio, con serie e ripetizioni specifiche.

La sezione **Sessioni** consente di pianificare allenamenti futuri. Una sessione contiene titolo, data, tipo, scheda associata, giorno della scheda, stato e note. Gli esercizi della sessione non vengono inseriti manualmente: sono ricavati automaticamente dal giorno scelto nella scheda. Questa scelta

rende la pianificazione più coerente, perché una sessione collegata a una scheda eredita il programma previsto.

La sezione **Storico** consente di registrare gli allenamenti svolti. Ogni allenamento contiene titolo, data, scheda associata, giorno svolto, durata, carico opzionale per singolo esercizio, fatica percepita da 1 a 10 e note. Quando viene registrato un allenamento collegato a una sessione prevista nello stesso giorno, la sessione passa automaticamente da "Da svolgere" a "Completata". Lo storico mostra anche un confronto semplice con l'allenamento precedente, basato sulla differenza di durata.

La sezione **Obiettivi** consente di creare, modificare, visualizzare ed eliminare obiettivi personali. Ogni obiettivo contiene titolo, descrizione, categoria, valore target, valore attuale, data di inizio, eventuale scadenza, stato e note. La categoria non è un testo libero, ma una scelta controllata: manuale, numero allenamenti, minuti totali, frequenza settimanale, carico su esercizio o completamento scheda. L'avanzamento viene mostrato tramite percentuale e barra di progresso. Per gli obiettivi automatici il valore attuale viene calcolato dagli allenamenti salvati, dai carichi registrati o dalle schede collegate. Quando il valore attuale raggiunge o supera il target, l'obiettivo viene considerato raggiunto. Se invece la data di fine viene superata senza raggiungere il target, l'obiettivo viene impostato come fallito.

La **Dashboard** mostra un riepilogo generale con numero di esercizi, numero di schede, sessioni nella settimana, allenamenti registrati, minuti totali di allenamento e obiettivi raggiunti. Include una card di ingresso rapido che propone la scheda programmata per oggi e apre direttamente la registrazione dell'allenamento. Mostra inoltre icone nei riquadri statistici, una riga di costanza settimanale colorata in base agli allenamenti registrati, la distribuzione compatta degli esercizi per gruppo muscolare e un timer di recupero.

2.2 Ricerca, filtri e organizzazione

Sono state implementate funzioni di ricerca e filtro in più sezioni:

- ricerca degli esercizi per nome;
- filtro degli esercizi per gruppo muscolare o difficoltà;
- ricerca delle schede per nome, obiettivo o livello;
- filtro delle schede per obiettivo o livello;
- ricerca delle sessioni per titolo, data, tipo, stato, scheda, giorno o esercizi;
- filtro delle sessioni per stato o tipo;
- ricerca nello storico allenamenti per titolo, data, note, scheda, giorno o esercizi;
- ricerca degli obiettivi per titolo, categoria o stato;
- filtro degli obiettivi per categoria o stato.

I filtri sono organizzati in una finestra dedicata con sezioni separate. Quando un filtro è attivo, l'app lo mostra esplicitamente all'utente.

2.3 Feature avanzate implementate

Sono state implementate due feature avanzate principali.

La prima è il **timer di recupero**, presente nella dashboard. L'utente può avviare o mettere in pausa il timer, scegliere rapidamente durate predefinite di 1:00 e 3:00 minuti, aumentare il tempo

con il pulsante **+30s** e diminuirlo con il pulsante **-30s**. Il timer è vincolato a un minimo di 30 secondi e a un massimo di 5 minuti, così da evitare valori non realistici o incrementi illimitati. La funzione è coerente con il dominio dell'app, perché il recupero tra le serie è una parte importante dell'allenamento.

La seconda è la **duplicazione di schede e sessioni**. L'utente può duplicare una scheda o una sessione esistente per creare rapidamente una variante. Il sistema genera nuovi identificativi e assegna un nome non duplicato, ad esempio aggiungendo "copia" o "copia 2".

Sono state inoltre implementate validazioni robuste sui dati, tramite l'utilizzo di Regex, migrazioni per dati salvati con versioni precedenti, inserimento automatico dei trattini nelle date, vincoli temporali per sessioni e allenamenti, conferme compatibili anche con la versione web e aggiornamenti automatici tra storico, sessioni e obiettivi.

2.4 Funzionalità considerate ma non implementate

Sono state considerate alcune funzionalità ulteriori, ma non inserite nella versione attuale:

- grafici avanzati sull'andamento dei carichi;
- calendario mensile completo;
- notifiche locali per ricordare le sessioni pianificate;
- autenticazione utente;
- sincronizzazione online;
- backend remoto;
- esportazione dei dati.

Queste funzionalità richiederebbero librerie aggiuntive o un'infrastruttura più complessa. Per questa versione si è scelto di mantenere l'app locale, stabile e coerente con i requisiti principali.

3 Progettazione dell'app

3.1 Struttura generale

L'app è organizzata in sei sezioni principali:

- Dashboard;
- Esercizi;
- Schede;
- Sessioni;
- Storico;
- Obiettivi.

La navigazione è gestita con React Navigation. Il progetto usa un **NavigationContainer**, uno Stack Navigator principale e un Bottom Tab Navigator per le sezioni dell'app. Questa scelta rispetta il vincolo della traccia sulla navigazione tra più schermate e rende la struttura più vicina a quella di una vera applicazione mobile.

La struttura principale del progetto è:

```
FitTrackPro/  
  App.tsx  
  index.ts  
  src/  
    components/  
      common.tsx  
      detail-modal.tsx  
      edit-modal.tsx  
      form-controls.tsx  
    modals/  
      detail/  
        exercise-detail-modal.tsx  
        goal-detail-modal.tsx  
        plan-detail-modal.tsx  
        session-detail-modal.tsx  
        workout-detail-modal.tsx  
      edit/  
        exercise-edit-modal.tsx  
        goal-edit-modal.tsx  
        plan-edit-modal.tsx  
        session-edit-modal.tsx  
        workout-edit-modal.tsx  
    screens/  
      dashboard-screen.tsx  
      exercises-screen.tsx  
      plans-screen.tsx  
      sessions-screen.tsx  
      workouts-screen.tsx  
      goals-screen.tsx  
    constants.ts  
    goals.ts  
    seed.ts  
    storage.ts  
    input-sanitizers.ts  
    styles.ts  
    types.ts  
    utils.ts  
    validation.ts
```

3.2 Schermate principali e flusso di navigazione

Il flusso principale parte dalla dashboard. L'utente può poi spostarsi tra le sezioni tramite la barra di navigazione inferiore.

Ogni sezione presenta un elenco di elementi e azioni coerenti:

- visualizzazione del dettaglio;
- creazione di un nuovo elemento;
- modifica di un elemento esistente;

- eliminazione con conferma;
- duplicazione dove prevista.

La visualizzazione del dettaglio e la modifica avvengono tramite modali. Questa scelta mantiene il flusso semplice: l'utente lavora sul dato rimanendo nel contesto della sezione corrente. Dal punto di vista progettuale le modali sono organizzate per entità: esercizi, schede, sessioni, allenamenti e obiettivi hanno file separati per inserimento/modifica e per dettaglio. I file **edit-modal.tsx** e **detail-modal.tsx** restano come dispatcher leggeri, cioè scelgono quale modale specifica aprire in base all'entità selezionata.

Schema sintetico della navigazione:

```
App
  NavigationContainer
    Stack Navigator
      MainTabs
        Dashboard
        Esercizi
        Schede
        Sessioni
        Storico
        Obiettivi
```

La navigazione dell'app non consiste soltanto nel passaggio tra le diverse sezioni tramite la barra inferiore, ma integra anche un flusso di interazione tra le varie funzionalità. Le schermate condividono infatti lo stesso stato principale dell'applicazione e le operazioni eseguite in una sezione possono produrre aggiornamenti immediatamente visibili anche nelle altre.

I principali flussi di navigazione sono i seguenti:

- **Accesso rapido dalla Dashboard.** La Dashboard rappresenta il punto di ingresso principale dell'app. Quando è presente una sessione pianificata per la giornata corrente, viene mostrata una card dedicata che consente di aprire direttamente la registrazione dell'allenamento. I dati della scheda selezionata, come esercizi, serie e ripetizioni, vengono precompilati automaticamente, evitando all'utente di reinserirli manualmente.
- **Gestione delle modali.** Le operazioni di inserimento, modifica e visualizzazione dei dettagli vengono eseguite tramite modali che si sovrappongono temporaneamente alla schermata corrente. Al termine dell'operazione, oppure in seguito alla chiusura della finestra, la modale viene rimossa e l'utente ritorna automaticamente alla schermata precedente, che mostra immediatamente i dati aggiornati.
- **Aggiornamento condiviso dei dati.** Lo stato principale dell'applicazione è gestito centralmente in **App.tsx**, mentre le schermate ricevono dati e funzioni tramite proprietà (props). Quando viene eseguita un'operazione, ad esempio la registrazione di un allenamento, la relativa callback aggiorna lo stato centrale. Di conseguenza tutte le schermate interessate vengono aggiornate automaticamente, consentendo ad esempio di visualizzare nella Dashboard le nuove statistiche e nella sezione Obiettivi l'eventuale avanzamento dei progressi.

3.3 Organizzazione dei dati

I tipi principali sono definiti in **src/types.ts**. Le entità gestite sono:

- **Exercise;**

- **WorkoutPlan;**
- **WorkoutPlanDay;**
- **PlannedSession;**
- **WorkoutLog;**
- **FitnessGoal.**

Tutti i dati sono raccolti nel tipo **FitnessData**, che contiene gli array delle entità. Ogni elemento ha un campo **id**, usato per modifica, eliminazione e collegamento con altri dati.

Il legame tra schede, sessioni, storico ed esercizi è basato sugli identificativi:

- una scheda contiene più giorni di allenamento;
- ogni giorno contiene esercizi collegati tramite **exerciseId**;
- ogni esercizio del giorno contiene serie e ripetizioni;
- una sessione contiene **planId** e **planDayId**;
- uno storico allenamento contiene **planId**, **planDayId** e carichi per singolo esercizio della scheda.
- un obiettivo può contenere riferimenti opzionali a un esercizio o a una scheda, usati per calcolare obiettivi come carico su esercizio o completamento scheda.

Questa scelta evita di duplicare oggetti complessi e mantiene i dati più coerenti. Se il nome di un esercizio cambia, le schede continuano a riferirsi allo stesso esercizio tramite identificativo.

3.4 Mockup e schermate dell'interfaccia

Per documentare l'interfaccia sono stati inseriti screenshot delle schermate principali dell'app. Gli screenshot non rappresentano wireframe preliminari, ma mockup dell'interfaccia finale: mostrano quindi il risultato effettivo dell'implementazione e aiutano a comprendere la disposizione dei contenuti, la navigazione e le principali azioni disponibili.

Oltre alle schermate interne, sono stati personalizzati anche gli asset grafici di avvio dell'applicazione: icona principale, splash screen, icona adattiva Android, versione monocromatica e favicon web. Questa scelta elimina gli asset predefiniti di Expo e rende l'app più riconoscibile come prodotto autonomo.

Le schermate incluse sono:

- dashboard principale, con ingresso rapido all'allenamento, riepiloghi, costanza settimanale e timer di recupero;
- elenco esercizi, con card, ricerca, filtri e azioni disponibili;
- modale di aggiunta esercizio, con campi controllati e selettori;
- sezione schede, con elenco delle schede di allenamento;
- sezione sessioni, dedicata alla pianificazione degli allenamenti;
- storico allenamenti, con riepilogo degli allenamenti svolti;
- sezione obiettivi, con stato di avanzamento e barra di progresso.

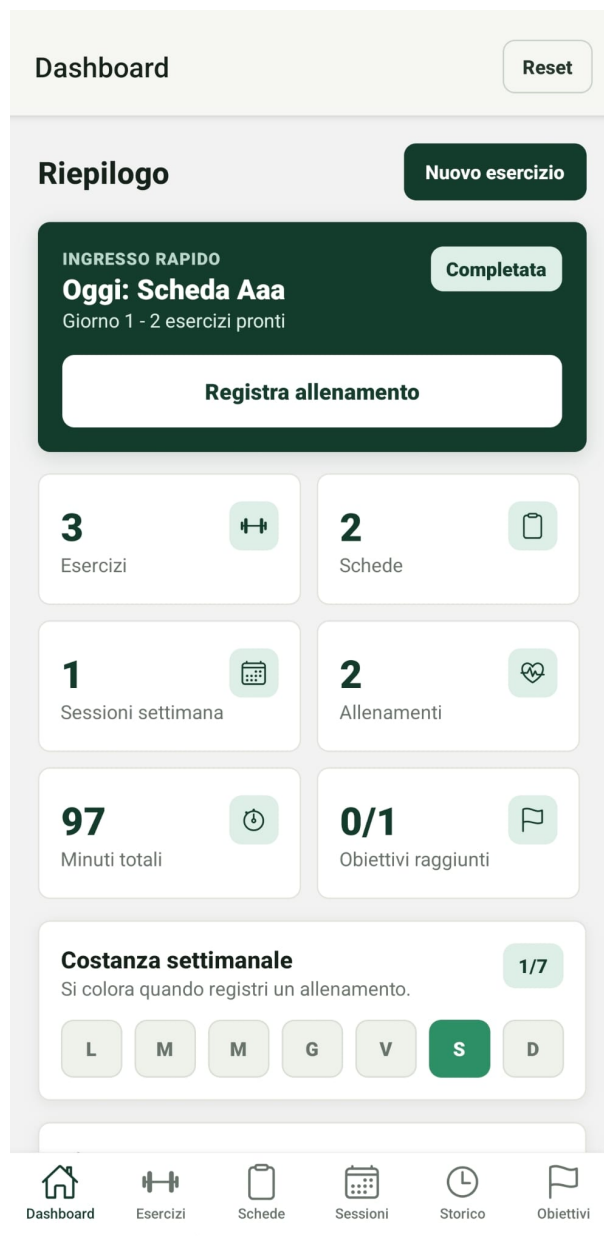


Figura 1: Dashboard principale

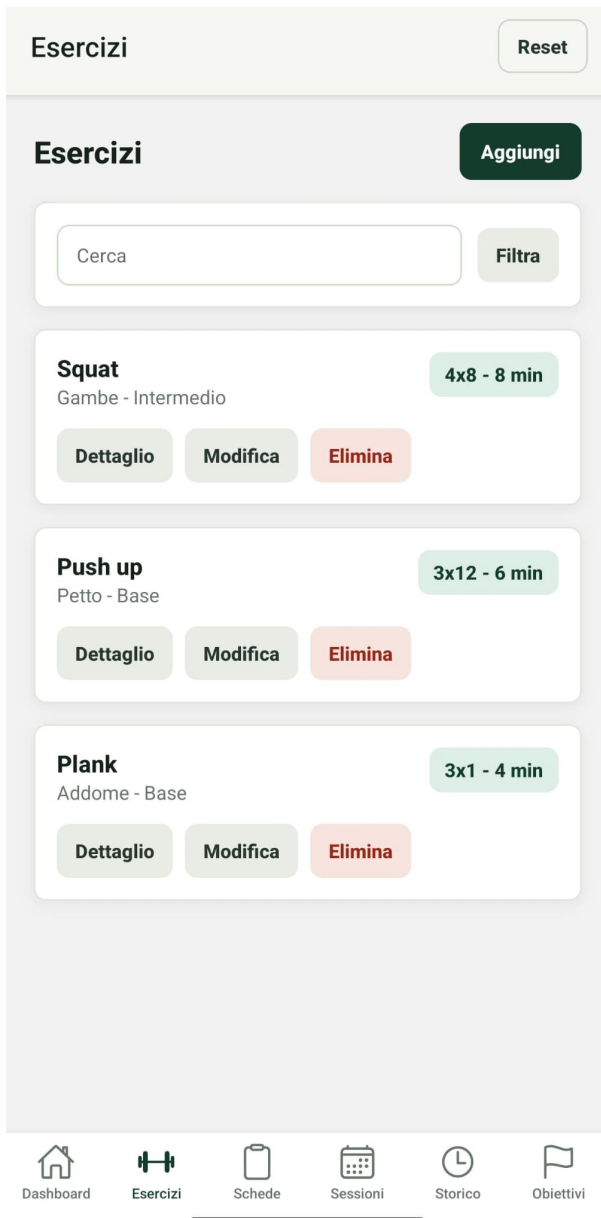


Figura 2: Elenco esercizi

Esercizio Chiudi

Nome *

Lettere, numeri, spazi, apostrofo e trattino. Massimo 60 caratteri.

Descrizione *

Testo libero controllato; caratteri invisibili e spazi anomali vengono rimossi. Massimo 500 caratteri.

Gruppo muscolare
Gambe Braccia Petto Schiena Addome
Spalle

Gruppi secondari
Braccia Petto Schiena Addome Spalle
Seleziona uno o più gruppi dalla lista.

Difficoltà
Base Intermedio Avanzato

Attrezzatura *
 Corpo libero
Testo breve controllato. Massimo 80 caratteri.

Serie consigliate (numero) *

Figura 3: Aggiunta esercizio

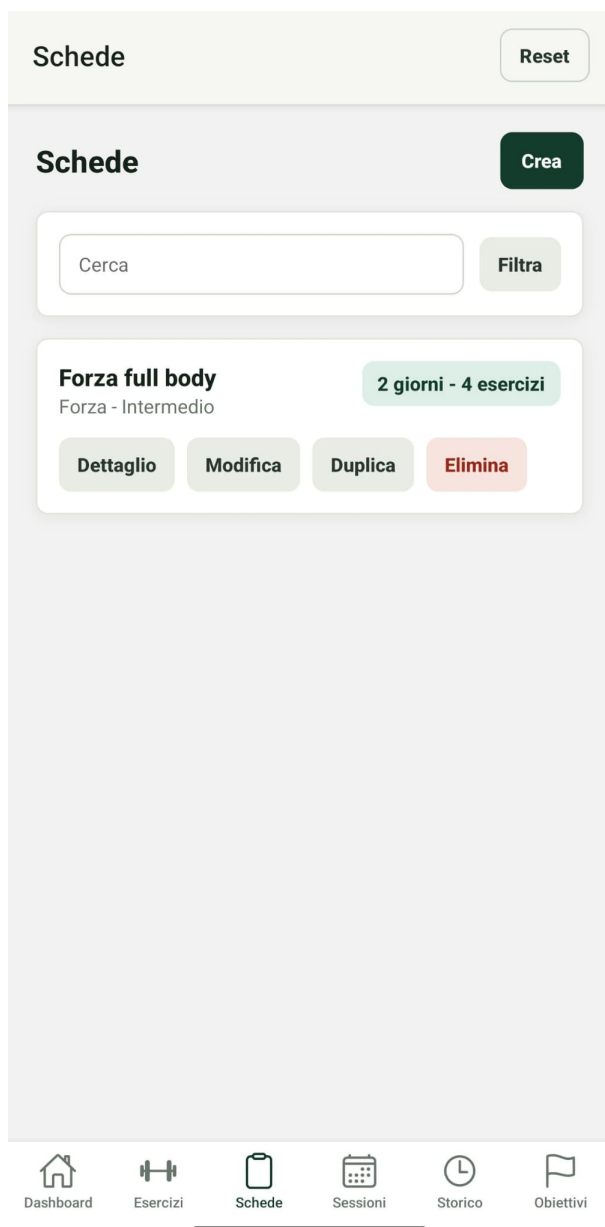


Figura 4: Schede di allenamento

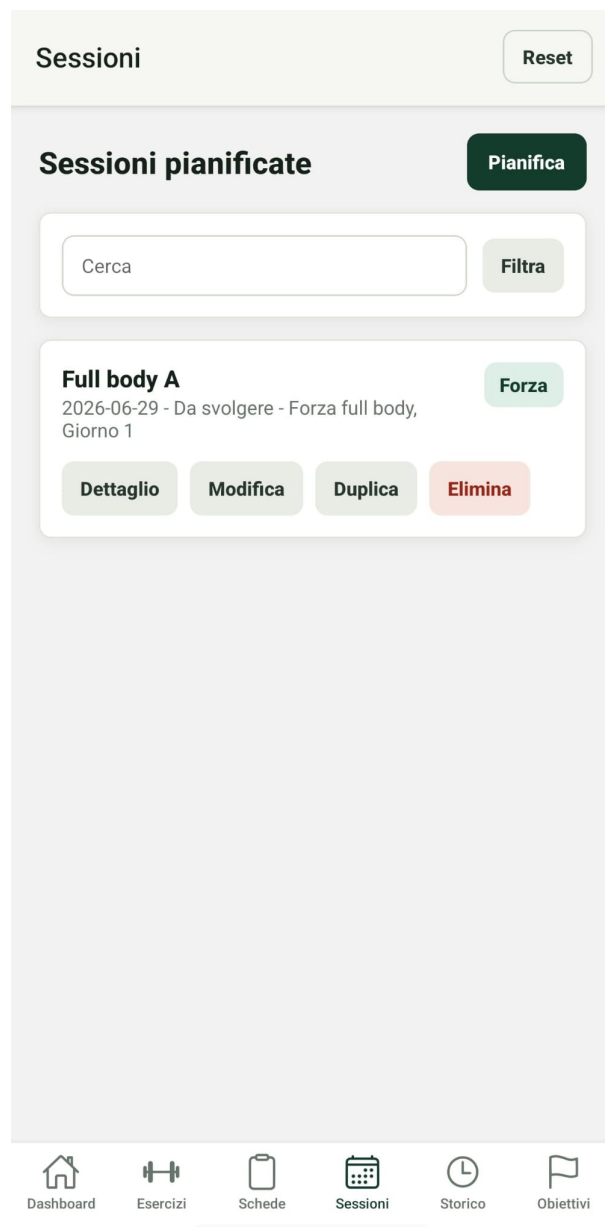


Figura 5: Sessioni pianificate



Figura 6: Storico allenamenti



Figura 7: Obiettivi personali

4 Scelte tecnologiche

4.1 Framework utilizzato

L'app è stata sviluppata con React Native tramite Expo. Le versioni principali, lette dal file `package.json`, sono:

- Expo SDK 54 (`expo ~54.0.0`);
- React Native 0.81.5;
- React 19.1.0;
- React DOM 19.1.0;
- TypeScript 5.9.2.

Expo è stato scelto per semplificare sviluppo, avvio e test su dispositivo fisico tramite Expo Go o su emulatore.

4.2 Librerie principali

Le dipendenze principali sono:

- `expo`;
- `react` 19.2.3;
- `react-dom` 19.1.0;
- `react-native` 0.85.3;
- `react-native-web` 0.21.0;
- `expo-status-bar` 3.0.9;
- `@react-navigation/native` 7.0.0;
- `@react-navigation/native-stack` 7.0.0;
- `@react-navigation/bottom-tabs` 7.0.0;
- `@react-native-async-storage/async-storage` 2.2.0;
- `@expo/vector-icons` 15.0.2;
- `react-native-screens` 4.16.0;
- `react-native-safe-area-context` 5.6.0.

React Navigation gestisce il passaggio tra schermate. AsyncStorage gestisce la persistenza locale. `@expo/vector-icons` viene usata per rendere più chiara la barra di navigazione inferiore tramite icone associate alle singole sezioni. Non sono usate API remote o backend, perché la traccia non li richiede.

4.3 Persistenza locale

La persistenza è implementata in `src/storage.ts`. All'avvio l'app prova a caricare i dati salvati localmente. Se non sono presenti dati, vengono caricati dati dimostrativi definiti in `src/seed.ts`.

Ogni modifica viene salvata in AsyncStorage serializzando l'oggetto `FitnessData` in JSON. Il file di storage contiene anche una normalizzazione dei dati, utile per mantenere compatibili dati salvati con strutture precedenti, ad esempio:

- schede vecchie senza giorni;
- sessioni vecchie senza giorno della scheda;
- storico vecchio con carico globale;
- gruppi muscolari vecchi come "Core" o "Addominali".
- obiettivi scaduti non completati, che vengono normalizzati nello stato "Fallito".

4.4 Motivazione delle scelte effettuate

La selezione dello stack tecnologico risponde alle esigenze funzionali dell'applicazione. React Native è stato scelto per consentire lo sviluppo di un'unica base di codice eseguibile su più piattaforme, riducendo il lavoro di sviluppo e manutenzione rispetto a soluzioni native separate. Inoltre, il paradigma dichiarativo di React semplifica la realizzazione dell'interfaccia utente e la gestione dei componenti. Expo è stato adottato per semplificare la configurazione dell'ambiente di sviluppo, facilitare i test su dispositivi mobili e consentire l'esecuzione dell'app anche tramite Expo Web.

Per la persistenza dei dati è stato utilizzato AsyncStorage, soluzione adeguata per un'applicazione che opera esclusivamente in locale e non richiede sincronizzazione con un backend. I dati vengono memorizzati come un unico oggetto JSON, evitando la complessità di un database relazionale e risultando sufficienti per il volume di informazioni gestito. Le relazioni tra le diverse entità sono mantenute tramite identificativi univoci e vengono ricostruite in memoria quando necessario, mentre statistiche e riepiloghi sono calcolati mediante gli hook di React. Questa scelta semplifica la gestione dei dati, riduce la complessità dell'implementazione e rende più agevoli le operazioni di migrazione e normalizzazione delle informazioni salvate.

5 Implementazione

5.1 Organizzazione del codice e Architettura Software

L'architettura dell'applicazione segue un approccio centralizzato per la gestione dello stato e una separazione netta tra i componenti di logica e quelli di presentazione.

Il file `App.tsx` funge da punto di ingresso (Entry Point) dell'applicazione, contenendo sia la configurazione della navigazione tramite React Navigation, sia la logica principale dell'app secondo il pattern dello *State Lifting* (sollevamento dello stato).

In particolare gestisce:

- la persistenza e lo stato dei dati dell'app;
- le statistiche calcolate in tempo reale;
- lo stato globale del timer globale per i recuperi;

- il collegamento rapido alla sessione d'allenamento del giorno;
- le funzioni di creazione e modifica delle entità;
- le operazioni di eliminazione sicura;
- la duplicazione rapida di schede e sessioni;
- lo stato di apertura e tracciamento dei dettagli;
- i meccanismi di ripristino e reset dei dati demo.

Le schermate ricevono dati e funzioni tramite props passate dai componenti route definiti in **App.tsx**. Questa scelta rende il flusso dei dati unidirezionale ed esplicito (pattern *Container-Presenter*): lo stato principale e le funzioni di mutazione risiedono in un solo file radice, mentre le schermate secondarie restano componenti puri di presentazione e interazione con l'utente.

La logica di basso livello e il core applicativo sono distribuiti in file di supporto specifici:

- **src/storage.ts** isola le chiamate asincrone a `AsyncStorage` e implementa gli algoritmi custom di migrazione e normalizzazione delle vecchie strutture dati;
- **src/input-sanitizers.ts** ospita le espressioni regolari (Regex) e le funzioni di pulizia preventiva delle stringhe per bloccare l'inserimento di caratteri di controllo ASCII non validi.

I componenti riutilizzabili dell'interfaccia utente sono organizzati all'interno della cartella **src/components**:

- **common.tsx** contiene gli elementi grafici atomici condivisi come card, pulsanti di azione, barre di ricerca, filtri ed elementi visivi per le statistiche;
- **form-controls.tsx** definisce i campi di input personalizzati, i gruppi di chip e i selettori, applicando automaticamente le funzioni di sanitizzazione durante la digitazione attiva dell'utente;
- **edit-modal.tsx** contiene il dispatcher leggero per l'apertura dinamica delle modali di inserimento e modifica;
- **detail-modal.tsx** contiene il dispatcher analogo dedicato alle schermate di visualizzazione del dettaglio;
- **src/components/modals/edit** ospita una modale di inserimento/modifica separata e specializzata per ogni singola entità dell'applicazione;
- **src/components/modals/detail** contiene una modale di dettaglio separata per ogni entità, condividendo un piccolo contenitore comune strutturato per gestire gli overlay nativi, le aree di scroll e il posizionamento del pulsante di chiusura;
- **goals.ts** isola il calcolo centralizzato dello stato di avanzamento e delle percentuali di completamento degli obiettivi.

Infine, tutti gli stili e le costanti grafiche sono centralizzati nel file **src/styles.ts**, garantendo una coerenza visiva rigorosa, l'applicazione di un tema scuro omogeneo e una manutenzione semplificata del layout tra le diverse sezioni.

5.2 Gestione dello stato

Lo stato è gestito con hook React: `useState`, `useEffect` e `useMemo`.

Gli stati principali sono:

- dati dell'app (`FitnessData`);
- elemento in modifica;
- elemento selezionato per il dettaglio;
- caricamento iniziale;
- timer di recupero;
- ricerca e filtro locali alle singole schermate.

`useEffect` viene usato per caricare i dati all'avvio e per gestire l'intervallo del timer. `useMemo` viene usato per calcolare statistiche derivate, come minuti totali, obiettivi raggiunti e distribuzione degli esercizi per gruppo muscolare.

5.3 Gestione della navigazione

La navigazione è gestita tramite React Navigation. L'app utilizza un `NavigationContainer` come contenitore principale, un `NativeStackNavigator` per la gestione della navigazione globale e un `BottomTabNavigator` per consentire il passaggio tra le sei sezioni principali. Questa struttura permette di mantenere separata la gestione della navigazione dalla logica applicativa e rende l'interfaccia semplice da utilizzare.

5.4 Gestione dei dati persistenti

La persistenza dei dati è gestita dal modulo `src/storage.ts`, che utilizza `AsyncStorage` per salvare e recuperare localmente tutte le informazioni dell'applicazione. Le principali operazioni sono implementate tramite due funzioni asincrone:

```
async function loadFitnessData(): Promise<FitnessData>;  
async function saveFitnessData(data: FitnessData): Promise<void>;
```

All'avvio dell'app viene richiamata la funzione di caricamento, che tenta di recuperare i dati precedentemente salvati. Se non sono presenti informazioni persistenti, il sistema inizializza automaticamente l'app utilizzando i dati dimostrativi definiti nel file `src/seed.ts`. Ogni modifica effettuata dall'utente viene successivamente serializzata in formato JSON e salvata nuovamente in `AsyncStorage`, garantendo la conservazione dello stato dell'app tra un'esecuzione e la successiva.

5.5 Inserimento, modifica e cancellazione

L'inserimento e la modifica avvengono tramite modali specifiche per entità. `EditModal` riceve lo stato dell'elemento in modifica e delega a `ExerciseEditModal`, `PlanEditModal`, `SessionEditModal`, `WorkoutEditModal` o `GoalEditModal`. In questo modo ogni file contiene solo la logica del proprio form, mentre il flusso di apertura e salvataggio rimane centralizzato.

La cancellazione richiede conferma. Su mobile viene usato **Alert**, mentre su web vengono usate le API native del browser tramite **globalThis.confirm**, così da mantenere lo stesso comportamento anche quando l'app viene eseguita con Expo Web.

La visualizzazione dei dettagli segue la stessa organizzazione: **DetailModal** delega a una modale specifica per l'entità selezionata. Questa separazione rende il codice più leggibile e riduce il rischio che una modifica agli obiettivi interferisca con schede, esercizi o allenamenti.

Il salvataggio passa dalla funzione di validazione centralizzata. Se il dato non è valido, l'app mostra un messaggio di errore e mantiene aperta la modale, permettendo all'utente di correggere l'input. Anche in questo caso il messaggio è gestito in modo compatibile con web e mobile.

5.6 Validazione e controllo degli input

Il controllo degli input opera su due livelli complementari.

Il primo livello è applicato durante la digitazione ed è centralizzato in **src/input-sanitizers.ts**. Il componente riutilizzabile **Field**, definito in **src/components/form-controls.tsx**, riceve una tipologia esplicita di input e filtra immediatamente il valore prima di aggiornare lo stato. In questo modo il controllo vale anche per i dati incollati e non dipende soltanto dalla tastiera visualizzata dal dispositivo.

Le tipologie gestite sono:

- **integer**: accetta esclusivamente cifre; segni **+** e **-**, lettere, punti, virgole e altri simboli vengono rimossi;
- **decimal**: accetta cifre e un solo separatore decimale, con massimo due cifre decimali; i segni non sono ammessi;
- **date**: accetta soltanto cifre e inserisce automaticamente i trattini nel formato **YYYY-MM-DD**;
- **name**: accetta lettere accentate, numeri, spazi, apostrofo, trattino e slash, così da permettere nomi composti come **Push/Pull**;
- **list**: accetta testo controllato e virgole per separare più elementi;
- **shortText**, **longText** e **search**: rimuovono caratteri invisibili, tabulazioni e simboli non pertinenti e applicano limiti di lunghezza.

Tutti i campi compilabili della modale dichiarano la propria tipologia. La barra di ricerca usa a sua volta la stessa sanitizzazione. **keyboardType** e **inputMode** migliorano l'esperienza d'uso, mentre il filtro applicato in **onChangeText** costituisce il controllo effettivo.

Il secondo livello è centralizzato in **src/validation.ts** e viene eseguito al salvataggio. Sono presenti controlli per evitare:

- campi obbligatori vuoti;
- nomi composti solo da spazi o caratteri speciali;
- duplicati su esercizi, schede, sessioni, allenamenti e obiettivi;
- date inesistenti, nel formato errato o non coerenti con il contesto d'uso;
- numeri negativi, decimali o fuori intervallo dove non consentiti;
- serie oltre 100 e ripetizioni oltre 10.000;

- durata oltre 1.440 minuti;
- frequenza settimanale oltre 14 e recupero oltre 3.600 secondi;
- carichi superiori a 1.000 kg o con più di due decimali;
- fatica percepita diversa da un intero compreso tra 1 e 10;
- target e valori attuali superiori a 1.000.000 o con più di due decimali;
- schede senza giorni, giorni senza esercizi e duplicazioni interne;
- riferimenti a schede, giorni o esercizi non esistenti;
- scadenze degli obiettivi precedenti alla data di inizio;
- test oltre i limiti previsti.

I test vengono inoltre normalizzati rimuovendo spazi iniziali, finali e sequenze multiple di spazi. Questa scelta evita duplicati apparenti, come "Squat" e " Squat ".

5.7 Gestione delle date

Le date sono salvate come stringhe nel formato **YYYY-MM-DD**. L'input inserisce automaticamente i trattini durante la digitazione: ad esempio **20260627** diventa **2026-06-27**.

La validazione non usa conversioni UTC tramite **toISOString**, per evitare errori dovuti al fuso orario. La data viene controllata confrontando anno, mese e giorno in locale.

Sono inoltre presenti vincoli specifici in base alla sezione: una sessione pianificata non può essere salvata con una data precedente al giorno corrente, mentre un allenamento registrato nello storico non può avere una data successiva al giorno corrente. In questo modo l'app distingue chiaramente tra pianificazione futura e registrazione di attività già svolte.

5.8 Riepiloghi e statistiche

La dashboard calcola e mostra:

- numero totale di esercizi;
- numero totale di schede;
- sessioni pianificate nella settimana;
- numero di allenamenti registrati;
- minuti totali di allenamento;
- obiettivi raggiunti rispetto al totale;
- costanza settimanale basata sulle date degli allenamenti registrati;
- distribuzione degli esercizi per gruppo muscolare.

Questi dati sono ricavati localmente dagli array salvati nell'app.

5.9 Aggiornamenti automatici tra entità

Alcune azioni dell'utente aggiornano più entità in modo coordinato. La registrazione di un allenamento può completare automaticamente la sessione pianificata corrispondente, quando data, scheda e giorno della scheda coincidono. Inoltre gli obiettivi automatici vengono ricalcolati partendo dagli allenamenti salvati nel periodo definito da data di inizio e scadenza.

Le categorie automatiche gestite sono: numero allenamenti, minuti totali, frequenza settimanale, carico su esercizio e completamento scheda. Gli obiettivi manuali restano aggiornabili dall'utente. Gli obiettivi vengono normalizzati sia al caricamento dei dati sia durante il salvataggio: se il valore attuale raggiunge il target diventano "Raggiunto"; se la data di fine è passata e il target non è stato raggiunto diventano "Fallito". Gli obiettivi falliti o raggiunti non vengono avanzati automaticamente da nuovi allenamenti.

6 Limitazioni note

L'app non utilizza backend remoto e quindi i dati restano sul dispositivo. In caso di disinstallazione dell'app o cancellazione dei dati locali, le informazioni possono andare perse.

Le statistiche sono volutamente semplici: non includono grafici avanzati sull'andamento dei carichi o analisi per periodo. Sono comunque coerenti con la richiesta di avere riepiloghi e indicatori visuali.

La pianificazione delle sessioni è collegata alle schede e ai giorni della scheda. Questo rende i dati coerenti, ma limita la possibilità di creare sessioni completamente libere senza scheda associata.

Gli obiettivi automatici coprono le categorie principali previste dall'app: allenamenti, minuti, frequenza, carichi su esercizio e completamento scheda. Obiettivi non riconducibili a queste categorie, ad esempio misure corporee o valutazioni personali, restano gestibili tramite categoria manuale.

7 Istruzioni per l'esecuzione

Per eseguire il progetto è necessario avere Node.js installato.

Dalla cartella del progetto:

```
cd percorso\FitTrackPro
npm install
npx expo start
```

L'app può essere aperta con Expo Go scansionando il QR code oppure tramite un emulatore Android/iOS configurato.

Per controllare che il codice TypeScript non contenga errori:

```
npx tsc --noEmit
```

Per controllare l'allineamento delle dipendenze Expo:

```
npx expo install --check
```

8 Verifiche effettuate

Durante il controllo finale sono state eseguite le seguenti verifiche:

- lettura della traccia progettuale;
- controllo della struttura del progetto;
- controllo delle dipendenze principali;
- verifica TypeScript con `npx tsc -noEmit`;
- verifica Expo con `npx expo install -check`;
- controllo dell'assenza di residui della precedente navigazione manuale;
- controllo dell'assenza di riferimenti a Expo Router, non usato nella versione finale;
- controllo dei campi **Field**, ciascuno con una tipologia di input esplicita;
- controllo dei soli due punti di creazione diretta di **TextInput**, entrambi protetti dalla sanitizzazione.

Le verifiche TypeScript ed Expo risultano superate.

9 Conclusioni

FitTrackPro soddisfa i requisiti principali della traccia: gestisce esercizi, schede di allenamento, sessioni pianificate, storico degli allenamenti e obiettivi personali. Include ricerca, filtri, riepiloghi, persistenza locale, navigazione tra più schermate e almeno due feature avanzate integrate nel dominio dell'app.

Le scelte più significative sono la navigazione con React Navigation, la gestione centrale dello stato in **App.tsx**, la modellazione delle schede in giorni di allenamento e il collegamento coerente tra schede, sessioni e storico. Il risultato è un'app usabile, organizzata e coerente con l'obiettivo di un fitness tracker personale.